



Session 5: SQL-TL5 - Creating and Managing Databases

Introduction

Welcome to Session 5 of the "Data Management with SQL Server" course! This session is all about learning how to create and manage databases in SQL Server 2022. Think of a database as a digital filing cabinet where you store and organize data, like customer records or sales information. We'll explore how to set up these "filing cabinets," organize their structure, and even take "snapshots" to save their state at a specific moment. We'll use SQL Server Management Studio (SSMS), a user-friendly tool, and Transact-SQL (T-SQL), a language for giving instructions to SQL Server. This guide is designed for beginners, so we'll explain everything step-by-step with examples, classwork, and tests to help you practice.

Learning Objectives

By the end of this session, you will:

- Understand how to safely modify system data (information about the database itself).
- Learn to create and manage filegroups (like folders for data) and transaction logs (records of changes).
- Know how to create and delete databases.
- Understand database snapshots and how they help save a point-in-time view of your data.

Session Content

1. Modification of System Data

Explanation for Beginners: System data is like the "blueprint" of your database, stored in special tables called system catalogs (e.g., `sys.databases`). These tables keep track of details like database names or settings. Changing this data is like updating the instructions for how your database works, but it's risky because mistakes can break things. Instead of editing these tables directly, we use safe commands like `ALTER DATABASE` to make changes.

- **Key Points:**

- System data includes metadata (data about data) like database settings or table structures.
- Avoid directly editing system tables to prevent errors or data loss.
- Use T-SQL commands or SSMS to update settings, like changing how the database handles recovery.

- **Example:**

Let's change a database's recovery model to `SIMPLE`, which reduces how much transaction history is saved (useful for test databases).

```
-- Change the recovery model of the SampleDB database to SIMPLE
ALTER DATABASE SampleDB
SET RECOVERY SIMPLE;
```

- **Code Explanation:**

- `ALTER DATABASE SampleDB` : Tells SQL Server we want to modify a database named `SampleDB`.
- `SET RECOVERY SIMPLE` : Changes the recovery model to `SIMPLE`, which means the transaction log (history of changes) is automatically cleared, saving space.

2. Adding Filegroups and Transaction Logs

Explanation for Beginners: A database is stored in files on your computer. Filegroups are like folders that organize these files, making it easier to manage large databases. Transaction logs are like a diary that records every change made to the database, so you can recover data if something goes wrong (e.g., a crash). Adding filegroups and logs helps improve performance and reliability.

- **Key Points:**

- Filegroups let you group data files for better organization or to spread data across multiple drives.
- Transaction logs ensure you can undo or redo changes if needed.
- Use `ALTER DATABASE` to add filegroups or additional log files.

- **Example:**

Add a new filegroup and a secondary data file to a database:

```
-- Add a new filegroup named SecondaryFG to SampleDB
ALTER DATABASE SampleDB
ADD FILEGROUP SecondaryFG;

-- Add a new data file to the SecondaryFG filegroup
ALTER DATABASE SampleDB
ADD FILE (
    NAME = 'SecondaryData', -- Logical name for the file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\Secondary',
    SIZE = 10MB, -- Initial size of the file
    MAXSIZE = 100MB, -- Maximum size the file can grow to
    FILEGROWTH = 5MB -- Amount the file grows when it needs more space
) TO FILEGROUP SecondaryFG;
```

Add a transaction log file:

```
-- Add a second transaction log file to SampleDB
ALTER DATABASE SampleDB
ADD LOG FILE (
    NAME = 'SampleDB_Log2', -- Logical name for the log file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SampleDB_Log2',
    SIZE = 5MB, -- Initial size of the log file
    MAXSIZE = 50MB, -- Maximum size for the log file
    FILEGROWTH = 5MB -- Growth increment for the log file
);
```

◦ Code Explanation:

- `ALTER DATABASE SampleDB` : Specifies the database to modify.
- `ADD FILEGROUP SecondaryFG` : Creates a new filegroup called `SecondaryFG` .
- `ADD FILE (...) TO FILEGROUP SecondaryFG` : Adds a new data file to the `SecondaryFG` filegroup, specifying its name, location, size, max size, and growth rate.
- `ADD LOG FILE (...)` : Adds a second transaction log file to store additional change history, with similar settings.

3. Creating a Database

Explanation for Beginners: A database is like a container for your data, holding tables,

views, and other objects. Creating a database is like setting up a new filing cabinet with specific rules for how data is stored. You can create it using T-SQL or the SSMS graphical interface.

- **Key Points:**

- Use the `CREATE DATABASE` command to set up a new database.
- Specify where data and log files are stored, their initial sizes, and how they grow.
- SSMS offers a point-and-click option to create databases.

- **Example:**

Create a database named `InventoryDB` :

```
-- Create a new database named InventoryDB
CREATE DATABASE InventoryDB
ON PRIMARY (
    NAME = 'InventoryDB_Data', -- Logical name for the primary data file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\Inventory',
    SIZE = 20MB, -- Initial size of the data file
    MAXSIZE = UNLIMITED, -- No limit on file size growth
    FILEGROWTH = 10MB -- File grows by 10MB when needed
)
LOG ON (
    NAME = 'InventoryDB_Log', -- Logical name for the transaction log file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\Inventory',
    SIZE = 10MB, -- Initial size of the log file
    MAXSIZE = 100MB, -- Maximum size of the log file
    FILEGROWTH = 5MB -- Log file grows by 5MB when needed
);
```

- **Code Explanation:**

- `CREATE DATABASE InventoryDB` : Starts the process to create a database named `InventoryDB` .
- `ON PRIMARY (...)` : Defines the primary data file, including its logical name, physical location, size, max size, and growth rate.
- `LOG ON (...)` : Defines the transaction log file with similar settings.

4. Dropping a Database

Explanation for Beginners: Dropping a database means deleting it completely, like throwing

away the entire filing cabinet. This removes all data and files, so be careful! You need to ensure no one is using the database before dropping it.

- **Key Points:**

- Use `DROP DATABASE` to delete a database and its files.
- Check for active connections (e.g., users or applications) before dropping.
- Always back up important data before dropping a database.

- **Example:**

Drop the `InventoryDB` database:

```
-- Delete the InventoryDB database and its associated files
DROP DATABASE InventoryDB;
```

- **Code Explanation:**

- `DROP DATABASE InventoryDB` : Permanently removes the `InventoryDB` database, including its data and log files.

5. Database Snapshots

Explanation for Beginners: A database snapshot is like taking a photo of your database at a specific moment. It's a read-only copy that lets you see the data as it was when the snapshot was created. This is great for reports or recovering data to that point in time.

- **Key Points:**

- Snapshots are read-only and don't allow changes.
- They use minimal disk space initially because they store only changes from the original database (using sparse files).
- Use `CREATE DATABASE ... AS SNAPSHOT OF` to create a snapshot.

- **Example:**

Create a snapshot of `SampleDB` :

```
-- Create a read-only snapshot of SampleDB named SampleDB_Snapshot
CREATE DATABASE SampleDB_Snapshot
AS SNAPSHOT OF SampleDB;
```

Revert `SampleDB` to the snapshot state:

```
-- Restore SampleDB to the state captured in SampleDB_Snapshot
RESTORE DATABASE SampleDB
FROM DATABASE_SNAPSHOT = 'SampleDB_Snapshot';
```

◦ **Code Explanation:**

- `CREATE DATABASE SampleDB_Snapshot AS SNAPSHOT OF SampleDB`: Creates a snapshot named `SampleDB_Snapshot` that captures the current state of `SampleDB`.
- `RESTORE DATABASE SampleDB FROM DATABASE_SNAPSHOT = 'SampleDB_Snapshot'`: Reverts `SampleDB` to the state it was in when the snapshot was created.

Classwork

These hands-on activities will help you practice what you've learned. Try them in SSMS.

1. **Create a Database:**

- **Task:** Create a database named `SalesDB` with a primary data file (50MB initial size, unlimited max size, 10MB growth) and a transaction log (20MB initial size, 200MB max size, 5MB growth).
- **T-SQL Code:**

```
-- Create a new database named SalesDB
CREATE DATABASE SalesDB
ON PRIMARY (
    NAME = 'SalesDB_Data', -- Logical name for the primary data file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SalesDB_Data.mdf',
    SIZE = 50MB, -- Initial size of the data file
    MAXSIZE = UNLIMITED, -- No limit on file size growth
    FILEGROWTH = 10MB -- File grows by 10MB when needed
)
LOG ON (
    NAME = 'SalesDB_Log', -- Logical name for the transaction log file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SalesDB_Log.ldf',
    SIZE = 20MB, -- Initial size of the log file
    MAXSIZE = 200MB, -- Maximum size of the log file
    FILEGROWTH = 5MB -- Log file grows by 5MB when needed
);
```

- **What You're Doing:** You're setting up a new database with a data file for storing tables and a log file for tracking changes.

2. Add a Filegroup and File:

- **Task:** Add a filegroup named `ArchiveFG` to `SalesDB` and a secondary data file (30MB initial size, 5MB growth). Verify using system views.

- **T-SQL Code:**

```
-- Add a new filegroup named ArchiveFG to SalesDB
ALTER DATABASE SalesDB
ADD FILEGROUP ArchiveFG;

-- Add a new data file to the ArchiveFG filegroup
ALTER DATABASE SalesDB
ADD FILE (
    NAME = 'SalesDB_Archive', -- Logical name for the file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\SalesDB_Archive.ndf',
    /size = 30MB, -- Initial size of the file
    MAXSIZE = UNLIMITED, -- No limit on file size growth
    FILEGROWTH = 5MB -- File grows by 5MB when needed
) TO FILEGROUP ArchiveFG;

-- Verify the filegroup exists
SELECT * FROM sys.filegroups;

-- Verify the file exists
SELECT * FROM sys.database_files;
```

- **What You're Doing:** You're organizing the database by adding a new filegroup and file, then checking the system catalogs to confirm they were added.

3. Create a Snapshot:

- **Task:** Create a snapshot of `SalesDB` named `SalesDB_Snapshot` and query it to confirm it reflects the current state.

- **T-SQL Code:**

```
-- Create a read-only snapshot of SalesDB
CREATE DATABASE SalesDB_Snapshot
AS SNAPSHOT OF SalesDB;

-- Query the snapshot to check its contents (assuming a table exists)
SELECT * FROM SalesDB_Snapshot.dbo.YourTable;
```

- **What You're Doing:** You're creating a snapshot to capture the current state of `SalesDB` and checking its data (replace `YourTable` with an actual table if available).

Test

These questions and tasks will test your understanding. Try answering without looking at the notes, then check your answers.

1. Multiple Choice Questions:

- **Question:** What is the purpose of a filegroup in SQL Server?
 - a) To store transaction logs only
 - b) To organize data files for performance and management
 - c) To encrypt database files
 - d) To back up the database
 - **Answer:** b
 - **Explanation:** Filegroups help organize data files, like putting files in different folders to improve performance or manage storage.
- **Question:** Which command deletes a database permanently?
 - a) DELETE DATABASE
 - b) DROP DATABASE
 - c) REMOVE DATABASE
 - d) ERASE DATABASE
 - **Answer:** b
 - **Explanation:** `DROP DATABASE` is the correct T-SQL command to delete a database and its files.

2. True/False:

- **Question:** Database snapshots are editable and can store new data.
 - **Answer:** False

- **Explanation:** Snapshots are read-only and only store a point-in-time view of the database.
- **Question:** Modifying system catalog tables directly is a recommended practice.
 - **Answer:** False
 - **Explanation:** Direct changes to system tables can corrupt the database; use commands like `ALTER DATABASE` instead.

3. Practical Task:

- **Task:** Write a T-SQL script to:
 - Create a database named `TestDB` with a 10MB data file and a 5MB log file.
 - Add a filegroup named `TestFG` and a 20MB data file to it.
 - Create a snapshot named `TestDB_Snapshot` .
- **Sample Solution:**

```

-- Create a new database named TestDB
CREATE DATABASE TestDB
ON PRIMARY (
    NAME = 'TestDB_Data', -- Logical name for the primary data file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestDB_Data.mdf',
    SIZE = 10MB, -- Initial size of the data file
    MAXSIZE = UNLIMITED, -- No limit on file size growth
    FILEGROWTH = 5MB -- File grows by 5MB when needed
)
LOG ON (
    NAME = 'TestDB_Log', -- Logical name for the transaction log file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestDB_Log.ldf',
    SIZE = 5MB, -- Initial size of the log file
    MAXSIZE = 50MB, -- Maximum size of the log file
    FILEGROWTH = 5MB -- Log file grows by 5MB when needed
);

-- Add a new filegroup named TestFG to TestDB
ALTER DATABASE TestDB
ADD FILEGROUP TestFG;

-- Add a new data file to the TestFG filegroup
ALTER DATABASE TestDB
ADD FILE (
    NAME = 'TestDB_Secondary', -- Logical name for the file
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\TestDB_Secondary.ndf',
    SIZE = 20MB, -- Initial size of the file
    MAXSIZE = UNLIMITED, -- No limit on file size growth
    FILEGROWTH = 5MB -- File grows by 5MB when needed
) TO FILEGROUP TestFG;

-- Create a read-only snapshot of TestDB
CREATE DATABASE TestDB_Snapshot
AS SNAPSHOT OF TestDB;

```

- **What You're Doing:** You're creating a database, adding a filegroup and file for organization, and taking a snapshot to save its state.

Deliverables Mapping

- **SQL Server - The Definitive Guide:**
 - **SG (Student Guide):** Session 5
 - **XP (Exercise Package):** Session 5
 - **TG (Trainer Guide):** Session 5

Additional Notes for Beginners

- **Getting Started with SSMS:** Open SSMS, connect to your SQL Server instance, and use the Query Editor to run T-SQL commands. Right-click on "Databases" in the Object Explorer to create a database using the GUI.
- **File Paths:** The file paths in the examples (e.g., `C:\Program Files\...`) are for SQL Server's default directory. Check your server's actual path in SSMS under "Server Properties > Database Settings."
- **Safety First:** Always back up important databases before dropping them. Use `BACKUP DATABASE SampleDB TO DISK = 'C:\Backups\SampleDB.bak';` to create a backup.
- **Practice Environment:** Use the AdventureWorks 2022 sample database (available from Microsoft) for extra practice, or create test databases like `SampleDB` or `TestDB`.
- **Common Errors:**
 - If you get a "file access denied" error, ensure you have permissions to the file path or run SSMS as an administrator.
 - If `DROP DATABASE` fails, check for active connections using `SELECT * FROM sys.dm_exec_sessions WHERE database_id = DB_ID('YourDB');`.

Tips for Success

- Run each T-SQL command one at a time in SSMS to see the results.
- Use `SELECT * FROM sys.databases;` to view all databases and confirm your work.
- If you're stuck, use SSMS's GUI to explore options (e.g., right-click a database to create a snapshot).
- Practice these commands multiple times to build confidence!